Machine Learning — Challenge 6
Advanced Unsupervised Learning: Anomaly Detection with AutoEncoders and Representation Learning

Prof. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Overview

Discover hidden structure in data without labels using clustering and representation learning techniques.

This challenge extends the unsupervised learning thread opened in Challenge 5 (clustering) into two advanced paradigms: **Anomaly Detection** via deep reconstruction models and **Representation Learning with Visualisation**. Both paradigms operate without labels and are therefore true unsupervised approaches, yet they produce qualitatively different insights: anomaly detection identifies *exceptional* data points, while representation learning reveals the *geometric structure* of the full data manifold.

Each group continues with the same domain and dataset used in Challenges 2 and 5. This continuity is intentional: by the end of Challenge 6, groups will have analysed the same real-world data through three complementary unsupervised lenses — semi-supervised signal extraction (Challenge 2), partition-based clustering (Challenge 5), and deep representation learning (this challenge). The final report must include a **cross-challenge synthesis** that compares what each paradigm revealed about the domain.

Learning objectives

Students will learn to:

- *Design* and train a **standard AutoEncoder (AE)** for tabular data and use reconstruction error as an unsupervised anomaly score.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

- *Implement* a **Variational AutoEncoder (VAE)** and understand how the probabilistic latent space differs from a deterministic one.
- *Apply* threshold-based anomaly detection: select a reconstruction-error cut-off using statistical methods (percentile, mean + $n\sigma$, or Isolation Forest as a cross-reference baseline).
- *Learn* low-dimensional representations of the data using the encoder and visualise them with **t-SNE** and **UMAP**.
- *Evaluate* anomaly detectors both intrinsically (error distributions, confusion analysis) and, where ground-truth signals exist, extrinsically (AUC-ROC, Average Precision).
- *Interpret* latent representations: compare cluster structure in latent space with the clusters found in Challenge 5 and discuss what the deep model captures that shallow clustering missed.
- *Communicate* findings in a scientific paper that synthesises all three unsupervised challenges on the same domain.

Challenge objective

Each group will:

1. Re-use the preprocessed feature matrix from Challenge 5 on the same domain dataset to ensure fair comparability across methods.
2. Train a **standard AutoEncoder** and use per-sample reconstruction error to rank data points from most-normal to most-anomalous.
3. Train a **Variational AutoEncoder** and study the structure of its latent space $\mathbf{z}$.
4. Visualise latent representations with **t-SNE** and **UMAP**; overlay cluster labels from Challenge 5 to cross-validate structure.
5. Train an **Isolation Forest** as a classical baseline and compare its anomaly rankings with those produced by the AE and VAE.
6. Produce a **cross-challenge synthesis section** in the paper comparing what K-Means/DBSCAN/Hierarchical (Challenge 5) and AE/VAE (this challenge) each revealed about the dataset.

Domain assignments (same as Challenges 2 and 5)

Groups retain the domain and dataset from the previous challenges. The **anomaly detection opportunity** specific to each domain is highlighted below.

1. **Group 1 — Transportation & Traffic Safety.** NHTSA FARS or BTS crash records. Reconstruct typical accident feature profiles; high reconstruction error identifies atypical incidents (e.g., multi-vehicle pile-ups under unusual weather). Visualise the latent space to study whether fatal vs. non-fatal crashes form separate manifold regions.

2. **Group 2 — Environment & Air Quality.** EPA AQS sensor measurements. Train the AE on normal-operation windows; anomalous reconstructions may correspond to pollution spikes or sensor faults. Use UMAP to visualise pollutant co-occurrence patterns in latent space.

3. **Group 3 — Public Health & Epidemiology.** CDC WONDER or BRFSS data. Detect counties or respondents whose health-behaviour profiles deviate strongly from the learned distribution. Explore whether the VAE latent space separates demographic or geographic subgroups not captured by Challenge 5 clusters.

4. **Group 4 — Education Systems.** NCES Common Core of Data. Identify schools or districts that are anomalous relative to peers with similar resource levels. Visualise the latent space to reveal equity-related structure (e.g., under-resourced high-performers).

5. **Group 5 — Economics & Socioeconomic Data.** ACS microdata from U.S. Census. Detect census tracts with anomalous income or employment profiles; these may correspond to gentrifying or economically declining areas. Compare the VAE latent clusters with the socioeconomic clusters from Challenge 5.

6. **Group 6 — Housing & Urban Development.** HUD or local assessor datasets. Reconstruct normal property/neighbourhood feature vectors; high-error properties may be data-entry errors or genuinely exceptional cases (e.g., historic properties, mixed-use outliers). Overlay UMAP embedding with housing-price quintiles.

7. **Group 7 — Crime & Public Safety.** FBI Crime Data Explorer or city open-data portals. Train the AE on typical incident feature vectors; anomalous reconstructions may correspond to rare crime types or unusual spatial/temporal patterns. Visualise temporal dynamics in the latent space.

8. **Group 8 — Energy & Utilities.** EIA consumption time-series or state utility datasets. AEs are particularly effective for time-series anomaly detection: reconstruct hourly or daily consumption windows and flag those with high error as potential outages or theft events. Visualise consumption regimes in the VAE latent space.

Required techniques

All three models are **mandatory**. They must be applied to the same preprocessed feature matrix used in Challenge 5.

Standard AutoEncoder (AE) for Anomaly Detection

An AutoEncoder is a neural network trained to *reconstruct its own input*:

$$\mathcal{L}_{\text{AE}} = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2, \tag{1}$$

where $\hat{\mathbf{x}}_i = \text{Decoder}(\text{Encoder}(\mathbf{x}_i))$. When trained only on *normal* patterns, the model learns a compressed representation of typical behaviour. At inference, a data point with

high reconstruction error is considered anomalous because it cannot be well-compressed by the encoder trained on normal data.

Architecture guidelines for tabular data:

- Use a *symmetric bottleneck* encoder/decoder: e.g., input $\to$ 128 $\to$ 64 $\to$ 16 (latent) $\to$ 64 $\to$ 128 $\to$ input.
- Use `ReLU` activations in hidden layers; no activation on the output layer (regression reconstruction).
- Train with `Adam` (lr $\approx 10^{-3}$) and `MSELoss` for 50–200 epochs; plot training loss to check convergence.

Anomaly scoring and threshold selection:

- Compute the per-sample reconstruction error $e_i = \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$ on the full dataset.
- Threshold strategies: 95th/99th percentile of e_i on the training set; mean + 3σ; or select the threshold that maximises F1 on a small validation set containing known anomalies (if any).
- Report the anomaly rate (fraction of samples above threshold) and inspect flagged samples qualitatively.

General implementation pattern (PyTorch):

```
import torch
import torch.nn as nn

class AutoEncoder(nn.Module):
    def __init__(self, input_dim, hidden_dims, latent_dim):
        super().__init__()
        enc_layers, dec_layers = [], []
        dims = [input_dim] + hidden_dims + [latent_dim]
        for i in range(len(dims) - 1):
            enc_layers += [nn.Linear(dims[i], dims[i+1]), nn.ReLU()]
        for i in range(len(dims) - 1, 0, -1):
            dec_layers += [nn.Linear(dims[i], dims[i-1]),
                           nn.ReLU() if i > 1 else nn.Identity()]
        self.encoder = nn.Sequential(*enc_layers[:-1])  # drop last ReLU
        self.decoder = nn.Sequential(*dec_layers)

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z), z

# --- Training ---
model = AutoEncoder(input_dim=X.shape[1],
```

```
                    hidden_dims=[128, 64], latent_dim=16)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()

for epoch in range(epochs):
    x_hat, _ = model(X_tensor)
    loss = criterion(x_hat, X_tensor)
    optimizer.zero_grad(); loss.backward(); optimizer.step()

# --- Anomaly scoring ---
with torch.no_grad():
    x_hat, _ = model(X_tensor)
    errors = ((X_tensor - x_hat) ** 2).mean(dim=1).numpy()

threshold = np.percentile(errors, 95)
anomalies = errors > threshold
```

Variational AutoEncoder (VAE) for Representation Learning

A Variational AutoEncoder learns a *probabilistic* latent space by encoding each input as a distribution $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ rather than a point. The model is trained to maximise the Evidence Lower BOund (ELBO):

$$\mathcal{L}_{\text{VAE}} = \underbrace{E_{q_\phi}\big[\log p_\theta(\mathbf{x}|\mathbf{z})\big]}_{\text{reconstruction}} - \underbrace{D_{\text{KL}}\big(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\big)}_{\text{regularisation}}, \tag{2}$$

where $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the prior. The KL term regularises the latent space into a smooth, continuous distribution, making it suitable for interpolation and visualisation.

Key differences from a standard AE:

- The encoder outputs $(\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2)$ rather than a single vector; a sample $\mathbf{z}$ is drawn via the *reparameterisation trick*: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- The loss has two terms: reconstruction (MSE) and KL-divergence. Balance via a β-VAE coefficient ($\beta = 1$ is standard; increase β to obtain more disentangled representations).
- Use $\boldsymbol{\mu}$ (not the sampled $\mathbf{z}$) as the deterministic representation for downstream visualisation and clustering.

General implementation pattern (PyTorch):

```
class VAE(nn.Module):
    def __init__(self, input_dim, hidden_dims, latent_dim):
        super().__init__()
        # Shared encoder body
        enc = []
        prev = input_dim
```

```
        for h in hidden_dims:
            enc += [nn.Linear(prev, h), nn.ReLU()]
            prev = h
        self.encoder_body = nn.Sequential(*enc)
        self.fc_mu    = nn.Linear(prev, latent_dim)
        self.fc_logvar = nn.Linear(prev, latent_dim)
        # Decoder
        dec = []
        prev = latent_dim
        for h in reversed(hidden_dims):
            dec += [nn.Linear(prev, h), nn.ReLU()]
            prev = h
        dec.append(nn.Linear(prev, input_dim))
        self.decoder = nn.Sequential(*dec)

    def encode(self, x):
        h = self.encoder_body(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterise(self, mu, logvar):
        std = (0.5 * logvar).exp()
        return mu + std * torch.randn_like(std)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterise(mu, logvar)
        return self.decoder(z), mu, logvar

def vae_loss(x_hat, x, mu, logvar, beta=1.0):
    recon = nn.functional.mse_loss(x_hat, x, reduction='sum')
    kld   = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return (recon + beta * kld) / x.size(0)
```

Visualisation with t-SNE and UMAP

Visualisation maps the high-dimensional latent representations $\{\mathbf{z}_i\}$ to 2-D (or 3-D) for human inspection.

- **t-SNE** (t-distributed Stochastic Neighbour Embedding): preserves local neighbourhood structure; perplexity controls the effective local neighbourhood size. Use `perplexity` $\in \{15, 30, 50\}$ and compare; t-SNE results vary between runs — fix `random_state`.
- **UMAP** (Uniform Manifold Approximation and Projection): faster than t-SNE and better preserves global structure; `n_neighbors` and `min_dist` are the key hyperparameters.

- Apply both t-SNE and UMAP to: (a) the *raw feature matrix* (as reference), (b) the *AE latent vectors*, and (c) the *VAE mean vectors* $\boldsymbol{\mu}$. Comparing these three embeddings shows what the deep models have learned.

General visualisation pattern:

```
from sklearn.manifold import TSNE
import umap  # pip install umap-learn

# t-SNE on VAE latent means
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
Z_tsne = tsne.fit_transform(Z_mu)   # Z_mu: numpy array (N, latent_dim)

# UMAP on AE latent vectors
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42)
Z_umap = reducer.fit_transform(Z_ae)

import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sc = axes[0].scatter(Z_tsne[:, 0], Z_tsne[:, 1],
                     c=cluster_labels_ch5, cmap='tab10', s=5)
axes[0].set_title('t-SNE of VAE latent space (Challenge 5 colours)')
axes[1].scatter(Z_umap[:, 0], Z_umap[:, 1],
                c=ae_errors, cmap='plasma', s=5)
axes[1].set_title('UMAP of AE latent space (coloured by anomaly score)')
plt.colorbar(sc, ax=axes[1])
plt.savefig('latent_visualisation.png', dpi=150)
```

Isolation Forest (classical baseline)

Isolation Forest is a tree-based anomaly detector that assigns each sample an *anomaly score* based on how quickly it is isolated in random binary partitions. It is conceptually simple, parameter-light (`contamination` controls the expected anomaly rate), and computationally efficient. Use it as a baseline to quantify how much the AE-based methods add beyond a classical detector.

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(n_estimators=200, contamination=0.05,
                      random_state=42)
iso_scores = -iso.fit(X_scaled).score_samples(X_scaled)
# Higher score => more anomalous (matches AE reconstruction error direction)
```

Compute the Spearman rank correlation between `iso_scores` and the AE/VAE reconstruction errors to understand how well the methods agree; disagreements are scientifically interesting to investigate.

Suggested workflow

1. **Data reuse and preprocessing.** Load the same preprocessed feature matrix from Challenge 5 (standardised, imputed, encoded). If the dataset has changed since Challenge 5, document all differences explicitly.

2. **Train/validation split.** Set aside 20% of the data as a *held-out test set*. Train the AE and VAE only on the *training portion*; compute reconstruction errors and latent representations for the full dataset (train + test) at evaluation time.

3. **AE training and anomaly scoring.** Train the standard AE, monitor the training curve, and compute per-sample reconstruction errors. Select the anomaly threshold via the 95th/99th percentile or mean + 3σ.

4. **VAE training and latent space extraction.** Train the VAE; extract the mean vectors $\boldsymbol{\mu}$ for all samples. Inspect the KL term during training to verify the latent space is regularised.

5. **Isolation Forest baseline.** Fit the Isolation Forest on the same preprocessed training data; compute anomaly scores for all samples.

6. **Comparison of anomaly detectors.** Rank all samples by each detector's score. Inspect the top-50 anomalies from each method: do they overlap? What domain properties do they share?

7. **Visualisation.** Apply t-SNE and UMAP to the AE latent vectors, the VAE mean vectors, and the raw feature matrix. Generate side-by-side scatter plots coloured by (a) anomaly score, (b) Challenge 5 cluster labels.

8. **Cross-challenge synthesis.** Write a comparative analysis connecting the findings from Challenges 2, 5, and 6. What structure does the AE latent space reveal that K-Means clusters did not? Do the anomalies detected here correspond to observations that were placed in small or outlier clusters in Challenge 5?

Evaluation metrics

- **Reconstruction error distribution**: plot the histogram of per-sample MSE; describe the shape (unimodal vs. bimodal) and the tail that defines anomalies.

- **Anomaly rate**: fraction of samples flagged by each method under the chosen threshold. Compare across AE, VAE reconstruction error, and Isolation Forest.

- **Detector agreement (Spearman ρ)**: rank correlation between AE errors, VAE errors, and Isolation Forest scores. Low agreement is informative and deserves analysis.

- **AUC-ROC / Average Precision (optional)**: if the domain dataset provides any proxy ground truth (e.g., verified crash fatalities in transportation, confirmed outages in energy), use it *only for post-hoc evaluation*. Never use ground truth during model training or threshold selection.

- **Latent space quality**: after projecting to 2-D with t-SNE/UMAP, compute the Silhouette Score on the latent representations using the Challenge 5 cluster labels as reference groups. A higher Silhouette Score on the latent space than on the raw feature space indicates that the AE/VAE learned a more cluster-friendly representation.
- **KL divergence during VAE training**: track the KL term separately from the reconstruction term in the training log; a collapsing KL (near zero) indicates posterior collapse and requires architectural adjustments.

Experimental design recommendations

- Train both AE and VAE with at least **three independent random seeds**; report mean and standard deviation of all metrics across seeds.
- Perform an **architecture ablation**: compare at least two bottleneck sizes (e.g., latent dimension 8 vs. 16 vs. 32) and report how latent dimension affects reconstruction quality and anomaly detection performance.
- For the VAE, perform a β **sweep** ($\beta \in \{0.5, 1, 4\}$) and visualise how β affects the structure of the latent space in the UMAP projection.
- Compare t-SNE and UMAP visualisations on both the *raw feature space* and the *AE/VAE latent spaces* to demonstrate what the models have learned beyond PCA compression.
- Include a **learning curve** (train loss per epoch) for both AE and VAE to show convergence and detect overfitting.

Common pitfalls (teaching moments)

- *Training on anomalies*: if the full dataset (including rare anomalous records) is used for training, the AE will learn to reconstruct anomalies too and lose its detection ability. Train only on records that are plausibly normal; when no labels are available, use the Isolation Forest to exclude the top-5% anomaly candidates from training.
- *Ignoring the KL term in VAE*: a reconstruction loss that is many orders of magnitude larger than the KL term will effectively disable regularisation. Normalise losses to the same scale or use β-VAE to balance them.
- *Posterior collapse*: if the VAE ignores the encoder and generates samples only from the prior, all latent means collapse to zero. Use a warm-up schedule: start with $\beta = 0$ and linearly ramp to $\beta = 1$ over the first 20–30 epochs.
- *t-SNE as ground truth*: t-SNE projections are sensitive to perplexity and random seed. Never interpret apparent cluster splits in t-SNE as definitive; always validate with quantitative metrics.
- *Threshold as a free parameter*: selecting the anomaly threshold by maximising a metric on the test set is label leakage. Fix the threshold on the training set (or on a small validation set if ground-truth signals exist) before evaluating on the test set.

- *Comparing reconstruction errors across models*: MSE scales with the number of input features and the model capacity. Normalise reconstruction errors before comparing AE vs. VAE on the same plot.
- *Neglecting the cross-challenge synthesis*: the most pedagogically valuable part of Challenge 6 is the comparative analysis. Budget significant report space for it.

Evaluation criteria

Submissions will be evaluated on the following aspects:

- **Implementation and architecture design (20%):** both AE and VAE are correctly implemented with appropriate architectures, training loops, and loss functions; Isolation Forest baseline is included.
- **Anomaly detection analysis (25%):** reconstruction error distributions are presented, thresholds are justified, top anomalies are characterised, and detector agreement (Spearman ρ) is reported and discussed.
- **Representation learning and visualisation (25%):** t-SNE and UMAP visualisations of AE, VAE, and raw feature spaces are included; latent space Silhouette Scores are compared; β and architecture ablations are reported.
- **Cross-challenge synthesis (20%):** a concrete, evidence-based comparison of findings from Challenges 2, 5, and 6 is presented in the paper; the discussion addresses what each paradigm revealed that the others did not.
- **Code quality, reproducibility, and video (10%):** seeds are fixed and recorded, environment is specified, a demonstration video is provided.

Expectations and experimental protocol

Groups must follow a rigorous protocol. At minimum, the following elements are required:

1. Use the **same preprocessed feature matrix** as Challenge 5 for all three models (AE, VAE, Isolation Forest) to ensure fair comparability.
2. Train AE and VAE with at least **three independent seeds**; report mean and standard deviation of reconstruction errors and metrics.
3. Produce the **six core figures**: (a) AE training loss curve, (b) VAE training loss curve (reconstruction + KL terms separately), (c) histogram of AE reconstruction errors with anomaly threshold marked, (d) t-SNE of VAE latent space coloured by anomaly score, (e) UMAP of AE latent space coloured by Challenge 5 cluster labels, (f) scatter plot comparing AE error vs. Isolation Forest score (with Spearman ρ).
4. Inspect and describe at least **10 specific anomalies** from the top-ranked samples: what are their domain properties? Are they data quality issues or genuine real-world outliers?

5. Include a **summary table** comparing AE, VAE, and Isolation Forest across all reported metrics.

Deliverables

- A public **GitHub repository** named `challenge-6_<groupid>` containing:
  - *Reproducible code*: clear `Jupyter notebook(s)` or scripts reproducing all experiments; provide a single *best-run* notebook and separate model module files (`autoencoder.py`, `vae.py`).
  - *Environment specification*: `requirements.txt` or `pyproject.toml`; a `Dockerfile` is strongly encouraged.
  - Exact commands or a `Makefile` to reproduce all reported results; record all random seeds.
  - *Saved model weights*: `.pt` checkpoint files for the best AE and VAE configurations.
  - *Figures*: all six core figures plus any additional visualisations saved as `.png` or `.pdf`.
  - *Results table*: a `.csv` or structured table of all metric values across model configurations and seeds.
- A **scientific paper** in IEEE format, maximum **seven (7) pages** excluding references. The paper must include: abstract, brief related work on deep anomaly detection and representation learning, data description (same as Challenges 2 and 5 for continuity), methodology (AE, VAE, and Isolation Forest architectures and training), experimental results (metrics, figures), anomaly case study, cross-challenge synthesis, and conclusions.
- A **short video** (7–10 minutes) uploaded to a public or unlisted link (e.g., OneDrive) demonstrating the project, the anomaly detection results, and a walkthrough of the latent space visualisations.

Notes on scope and computation

- For large datasets (>100 000 records), train on a stratified sample of 50 000 records and compute errors/representations on the full dataset.
- Training an AE and VAE on tabular data with 10–50 features is fast on CPU (minutes per run); GPU is not required but will accelerate experiments if available.
- `umap-learn` must be installed separately (`pip install umap-learn`); it is not part of `scikit-learn`.
- t-SNE is $\mathcal{O}(N^2)$ in memory for exact computation; use `method='barnes_hut'` (default in scikit-learn for $N > 1000$) or apply t-SNE to a random subsample of 10 000 points when the full dataset is larger.

Reproducibility checklist

- Fix and record all `random_state` / `seed` values (Python, NumPy, PyTorch / TensorFlow).
- Provide environment specification (`requirements.txt` or `pyproject.toml`). Docker is encouraged but optional.
- Include run scripts or `Makefile` for ease of evaluation.
- Confirm that the preprocessing pipeline is identical to Challenge 5 and document any deviations.

Grading checklist (to include in the repository)

Each submission must include a short `CHECKLIST.md` containing:

- The dataset name, source URL, number of records, and number of features (same as Challenge 5 or documented deviations).
- AE and VAE final architectures (layer sizes, latent dimension, activation functions, number of training epochs, learning rate, and β value for VAE).
- The chosen anomaly threshold for each method (AE, VAE, Isolation Forest) and the resulting anomaly rate.
- The Spearman rank correlation between AE, VAE, and Isolation Forest anomaly scores.
- The Silhouette Score on AE latent representations using Challenge 5 cluster labels, compared with the Silhouette Score on the raw feature space.
- A short paragraph (max 200 words) summarising what Challenge 6 revealed about the domain that Challenges 2 and 5 did not.

References and further reading

Include canonical references in the IEEE paper: the original AutoEncoder literature (Hinton & Salakhutdinov, 2006), the VAE paper (Kingma & Welling, 2014), the β-VAE (Higgins et al., 2017), t-SNE (van der Maaten & Hinton, 2008), UMAP (McInnes et al., 2018), Isolation Forest (Liu et al., 2008), and at least one recent survey on deep anomaly detection. All citations must be in IEEE style.

Challenge 6 is the culmination of the unsupervised learning arc. Students who complete it will have applied three distinct paradigms to the same real-world dataset and will have developed the critical judgment to articulate what each paradigm sees, what it misses, and why that matters for the domain they studied.